

Data Analysis of XRD, BET and TGA-DTA/DTG in MATLAB

Complete Handbook with Codes, Explanations, and References

Bianca Maciel Marques

Postgraduate Program in Materials Engineering and Science (PIPE)

Federal University of Paraná

`bianca.marques@ufpr.br`

July 21, 2026

Abstract

This handbook provides a practical and in-depth guide for analyzing X-ray diffraction (XRD) data, including Rietveld refinement and post-analysis, surface area determination by the BET method, and thermogravimetric analysis (TGA-DTA/DTG) using MATLAB. The material is structured for researchers who wish not only to run codes but also to understand the mathematical, statistical, and physical fundamentals of each technique. Codes with error checking, automatic table and figure saving, and detailed explanations of each processing step from data reading to publication-quality graph customization are presented.

Contents

1	General Introduction	5
1.1	Handbook Structure	5
2	XRD Analysis (X-Ray Diffraction)	6
2.1	Theoretical Foundations	6
2.1.1	Bragg's Law	6
2.1.2	Scherrer Equation	6
2.1.3	Williamson-Hall Method	7
2.2	The Rietveld Method and Its Practical Implementation	7
2.2.1	Principle of the Method	7
2.2.2	Profile Function (Pseudo-Voigt)	7
2.2.3	Refined Parameters in Rietveld	7
2.2.4	Quality Indices of Refinement	8
2.2.5	MATLAB's Role in the Rietveld Process	8
2.3	Input Data Structure	8
2.4	MATLAB Code for XRD With Detailed Explanations	9
2.4.1	Code Overview	9
2.4.2	Detailed Code Explanation	13
2.5	Example Input and Output	14
2.5.1	Input (ZIF-8_400mL.xy)	14
2.5.2	Output Table XRD_results.xlsx	14
2.5.3	Output Williamson-Hall	14
2.6	Graph Customization for Publication	14
3	BET Analysis (Specific Surface Area)	17
3.1	Theoretical Foundations	17
3.1.1	BET Equation	17
3.1.2	BET Parameters	17
3.1.3	Specific Surface Area	17
3.1.4	Validity Range and Rouquerol Criteria	18
3.2	MATLAB Code for BET With Detailed Explanations	18
3.3	Detailed Code Explanation for BET	20
3.4	Example Input and Output	21
3.4.1	Input (ZIF-8_400mL_isotherm.txt)	21
3.4.2	Output Table BET_results.xlsx	21
4	TGA-DTA/DTG Analysis (Thermogravimetry)	22
4.1	Theoretical Foundations	22
4.1.1	Mass Loss	22
4.1.2	DTG (Derivative of TGA)	22
4.1.3	Onset Temperature	22
4.1.4	Decomposition Kinetics Kissinger Method	22
4.2	MATLAB Code for TGA With Detailed Explanations	22
4.3	Detailed Code Explanation for TGA	26
4.4	Example Input and Output	27
4.4.1	Input (ZIF-8_400mL_TGA.txt)	27

4.4.2	Output Table TGA_loss_results.xlsx	27
4.4.3	Output TGA_onset_results.txt	27
5	Multiple File Plotting and Customization	29
5.1	Example Code for Overlay Plotting	29
5.2	Advanced Customization Table	30
5.3	Tips for Publication-Quality Graphs	30
A	Summary of Generated Files	31

1 General Introduction

Materials characterization is a cornerstone of research in chemistry, physics, and materials engineering. Three techniques are particularly ubiquitous in research laboratories: X-ray diffraction (XRD), gas adsorption (BET), and thermogravimetric analysis (TGA). Each provides complementary information about crystal structure, texture, and thermal behavior of materials.

MATLAB is a powerful tool for processing and visualizing experimental data. Although specialized software exists for each of these techniques (such as GSAS/FullProf for Rietveld, or the equipment's own software for BET), MATLAB offers the flexibility to customize analyses, automate repetitive tasks, integrate different methods, and generate graphs with typographic precision for publications.

This handbook has been developed with the following objectives:

- **Deep understanding:** Explain the mathematical and physical foundations of each technique.
- **Robust implementation:** Provide MATLAB codes with error checking, exception handling, and automatic result saving.
- **Didactic explanation:** Each code block is accompanied by a detailed explanation of what it does and why.
- **Customization:** Show how to adjust colors, fonts, line styles, and export figures for articles.

1.1 Handbook Structure

The handbook is organized into three main chapters, each dedicated to a characterization technique:

1. **XRD:** From Bragg's Law to Rietveld refinement, including Scherrer and Williamson-Hall.
2. **BET:** From adsorption isotherm to surface area, with statistical validation.
3. **TGA:** From mass loss to decomposition kinetics, with correct plateau-based loss calculation.

Each chapter contains: theoretical foundations, input data structure, complete MATLAB code, line-by-line explanation, example results, and references.

2 XRD Analysis (X-Ray Diffraction)

2.1 Theoretical Foundations

X-ray diffraction is a non-destructive technique that allows identification of crystalline phases, determination of unit cell parameters, calculation of crystallite size, and evaluation of microstrains. When a monochromatic X-ray beam strikes a crystalline material, atoms scatter the radiation. In certain directions, interference between rays scattered by different atomic planes is constructive, generating diffraction peaks.

2.1.1 Bragg's Law

The fundamental condition for diffraction is Bragg's Law, which relates wavelength (λ), interplanar spacing (d), and incidence angle (θ):

$$n\lambda = 2d \sin \theta \quad (1)$$

where n is the diffraction order (usually 1). From the peak positions in 2θ , we can calculate d for each hkl plane set, thus identifying the phases present (comparison with databases like ICDD PDF).

2.1.2 Scherrer Equation

The width of diffraction peaks is inversely related to crystallite size. Smaller crystallites produce broader peaks due to incomplete diffraction. The Scherrer equation quantifies this relationship:

$$D = \frac{K\lambda}{\beta \cos \theta} \quad (2)$$

where:

- D is the average crystallite size (in nm after conversion)
- K is the shape constant (0.9 for spherical particles)
- λ is the wavelength (e.g., 1.5406 Å for Cu K)
- β is the full width at half maximum (FWHM) in radians, corrected for instrumental broadening
- θ is the Bragg angle in radians

Instrumental broadening correction is essential. The observed FWHM (β_{obs}) is a convolution of the sample contribution (β_{sample}) with instrumental contribution (β_{inst}). For Gaussian profiles, deconvolution is done by quadratic subtraction:

$$\beta_{sample} = \sqrt{\beta_{obs}^2 - \beta_{inst}^2} \quad (3)$$

The instrumental broadening varies with angle and can be modeled by the Caglioti equation:

$$H_{inst}^2 = U \cdot \tan^2(\theta) + V \cdot \tan(\theta) + W \quad (4)$$

where U , V , and W are parameters obtained from a standard measurement (e.g., silicon).

2.1.3 Williamson-Hall Method

The Williamson-Hall method allows separation of crystallite size (D) and microstrain (ε) contributions to peak broadening. The fundamental equation is:

$$\beta \cos \theta = \frac{K\lambda}{D} + 4\varepsilon \sin \theta \quad (5)$$

Fitting a line between $\beta \cos \theta$ (y-axis) and $4 \sin \theta$ (x-axis):

- The intercept gives $\frac{K\lambda}{D}$ (crystallite size)
- The slope gives ε (microstrain)

This method is more robust than Scherrer alone because it uses multiple peaks and provides an estimate of lattice strain.

2.2 The Rietveld Method and Its Practical Implementation

The Rietveld method, proposed by Hugo Rietveld in 1969, revolutionized powder crystallography. It is a least-squares refinement method that fits a complete structural model to the observed diffraction pattern.

2.2.1 Principle of the Method

In the Rietveld method, the calculated intensity at each point i of the pattern is given by:

$$Y_{calc}(2\theta_i) = Y_{bg}(2\theta_i) + \sum_k I_k \cdot \Omega(2\theta_i - 2\theta_k) \quad (6)$$

where:

- Y_{bg} is the background intensity, modeled by a polynomial or Chebyshev function
- I_k is the integrated intensity of peak k
- Ω is the profile function describing peak shape

2.2.2 Profile Function (Pseudo-Voigt)

The peak shape is a linear combination of a Gaussian (G) and Lorentzian (L) function:

$$\Omega = \eta \cdot L + (1 - \eta) \cdot G \quad (7)$$

where η is the mixing parameter ($0 \leq \eta \leq 1$). The Gaussian describes broadening due to crystallite size, while the Lorentzian describes broadening due to microstrain and defects.

2.2.3 Refined Parameters in Rietveld

The parameters adjusted in the refinement can be grouped into:

1. **Unit cell parameters:** $a, b, c, \alpha, \beta, \gamma$
2. **Profile parameters:** U, V, W (Caglioti) and η (mixing)

3. **Background parameters:** polynomial coefficients
4. **Structural parameters:** atomic positions (x, y, z), occupancy factors, isotropic temperature factors (B_{iso})
5. **Sample parameters:** preferred orientation, asymmetry, zero shift

2.2.4 Quality Indices of Refinement

Two factors are universally used to evaluate refinement quality:

1. R_{wp} (weighted R):

$$R_{wp} = \left[\frac{\sum_i w_i (Y_{obs,i} - Y_{calc,i})^2}{\sum_i w_i Y_{obs,i}^2} \right]^{1/2} \times 100\% \quad (8)$$

Values below 10% are considered good for inorganic materials.

2. GOF (Goodness of Fit):

$$GOF = \sqrt{\frac{\sum_i w_i (Y_{obs,i} - Y_{calc,i})^2}{N - P}} \quad (9)$$

where N is the number of points and P is the number of refined parameters. A GOF close to 1 indicates an ideal fit (error overestimation gives $GOF < 1$; underestimation gives $GOF > 1$).

2.2.5 MATLAB's Role in the Rietveld Process

Important: MATLAB is **not** used to perform Rietveld refinement itself. The numerical refinement is done in specialized software such as **GSAS-II**, **FullProf**, **MAUD**, or **Topas**, which have robust non-linear least-squares algorithms with dozens of coupled parameters.

MATLAB is used for:

1. **Post-processing:** Reading refinement output files and generating high-quality graphs (Observed, Calculated, Difference, Bragg markers).
2. **Index calculation:** Computing R_{wp} and GOF directly from output files, avoiding transcription errors.
3. **Comparison:** Overlaying multiple refinements for comparative analysis.

2.3 Input Data Structure

Diffraction file: Text file with two columns separated by spaces/tabs:

- Column 1: 2θ angle in degrees
- Column 2: intensity (counts)

Example `diffraction.xy`:

```
10.00    120
10.05    135
10.10    142
...
70.00    210
```


Files for Rietveld plot (after refinement):

- rietveld_result.txt: three columns (2θ , Y_{obs} , Y_{calc})
- bragg_positions.txt: one column with Bragg peak 2θ angles

2.4 MATLAB Code for XRD With Detailed Explanations**2.4.1 Code Overview**

The following code performs: 1. Reads the data file. 2. Detects peaks using findpeaks. 3. For each peak, fits a Gaussian to extract FWHM. 4. Calculates interplanar spacing d by Bragg's Law. 5. Calculates crystallite size by Scherrer equation, correcting for instrumental broadening via Caglioti. 6. Performs Williamson-Hall fitting for size and microstrain. 7. Generates graphs: diffractogram with marked peaks, Williamson-Hall plot, and Rietveld plot (if files exist). 8. Saves tables (Excel/CSV) and figures (PNG/PDF).

```

1 %% =====
2 % XRD ANALYSIS - d-SPACING, CRYSTALLITE SIZE,
3 % MICROSTRAIN AND RIETVELD PLOTTING
4 % =====
5
6 clear; clc; close all; % Clear workspace, screen, and figures
7
8 % ===== CONFIGURATIONS (USER MUST ADJUST) =====
9 xrd_file = 'ZIF-8_400mL.xy'; % Data file name
10 lambda = 1.5406; % Wavelength (Cu K) in Å
11 % Caglioti parameters (obtained from Si standard)
12 U = 0.01; V = -0.005; W = 0.005; % in rad^2
13 K = 0.9; % Shape constant (Scherrer)
14 rietveld_file = 'rietveld_result.txt'; % GSAS/FullProf output
15 bragg_file = 'bragg_positions.txt'; % Bragg peaks
16 % =====
17
18 % Check if file exists; otherwise, stop with error
19 if ~exist(xrd_file, 'file')
20     error('File %s not found.', xrd_file);
21 end
22
23 % Load data: two columns (2theta, intensity)
24 data = load(xrd_file);
25 two_theta = data(:, 1);
26 intensity = data(:, 2);
27
28 % ===== PEAK DETECTION =====
29 % findpeaks finds local maxima with minimum height of 20% of max peak
30 % and minimum distance of 2 degrees between peaks
31 [peak_int, peak_idx] = findpeaks(intensity, two_theta, ...
32     'MinPeakHeight', max(intensity)*0.2, ...
33     'MinPeakDistance', 2.0); % Distance in degrees (using X vector)
34
35 peak_2theta = two_theta(peak_idx);
36
37 % Preallocate result vectors
38 d_vals = zeros(length(peak_2theta), 1);
39 D_vals = zeros(length(peak_2theta), 1);
40 FWHM_rad = zeros(length(peak_2theta), 1);

```

```

41
42 % ===== EACH PEAK FITTED BY A GAUSSIAN =====
43 % Method: uses lsqcurvefit (Optimization Toolbox) for fitting
44 % Model: Gaussian + linear background: p(1)+p(2)*x +
      p(3)*exp(-(x-p(4))^2/(2*p(5)^2))
45 % p(1)=offset, p(2)=background slope, p(3)=height, p(4)=center,
      p(5)=sigma
46 for i = 1:length(peak_2theta)
47     % Window of ±1.5 degrees around the peak
48     idx_window = (two_theta >= peak_2theta(i)-1.5) & (two_theta <=
        peak_2theta(i)+1.5);
49     x_window = two_theta(idx_window);
50     y_window = intensity(idx_window);
51
52     % If too few points, skip this peak
53     if length(x_window) < 10
54         warning('Peak %d: too few points for fitting.', i);
55         continue;
56     end
57
58     % Define Gaussian model with linear background
59     gaussModel = @(p, x) p(1) + p(2)*x +
        p(3)*exp(-(x-p(4)).^2)/(2*p(5)^2));
60
61     % Initial guesses: [offset, slope, height, center, sigma]
62     p0 = [min(y_window), 0, max(y_window)-min(y_window), peak_2theta(i),
        0.2];
63     opts = optimoptions('lsqcurvefit', 'Display', 'off');
64
65     % Perform fitting; if it fails, skip this peak
66     try
67         p_fitted = lsqcurvefit(gaussModel, p0, x_window, y_window, [],
            [], opts);
68     catch
69         warning('Fitting failed for peak %d.', i);
70         continue;
71     end
72
73     % Extract sigma and calculate FWHM in radians
74     sigma = abs(p_fitted(5));
75     FWHM_rad(i) = 2 * sqrt(2*log(2)) * sigma * pi/180; % Convert
        degrees -> rad
76
77     % ===== INSTRUMENTAL CORRECTION (CAGLIOTI) =====
78     theta_rad = deg2rad(peak_2theta(i)/2);
79     % Instrumental broadening in radians (Caglioti equation)
80     H_inst_rad = sqrt(U * tan(theta_rad)^2 + V * tan(theta_rad) + W);
81     % Gaussian deconvolution (quadratic subtraction)
82     FWHM_corrected_rad = sqrt(max(FWHM_rad(i)^2 - H_inst_rad^2, 0));
83
84     % ===== d-SPACING CALCULATION (BRAGG'S LAW) =====
85     d_vals(i) = lambda / (2 * sin(theta_rad)); % in Å
86
87     % ===== CRYSTALLITE SIZE (SCHERRER) =====
88     D_vals(i) = (K * lambda) / (FWHM_corrected_rad * cos(theta_rad)) /
        10; % in nm
89 end
90

```

```

91 % ===== WILLIAMSON-HALL =====
92 % Use only peaks with valid FWHM (>0)
93 valid = FWHM_rad > 0;
94 if sum(valid) >= 3 % Need at least 3 peaks for linear fitting
95     % Transformed variables: x = 4*sin(theta), y = beta*cos(theta)
96     x_WH = 4 * sin(deg2rad(peak_2theta(valid)/2));
97     y_WH = FWHM_rad(valid) .* cos(deg2rad(peak_2theta(valid)/2));
98     p_WH = polyfit(x_WH, y_WH, 1); % Linear fit (y = a*x + b)
99     D_WH = (K * lambda) / p_WH(2) / 10; % nm (intercept = K*lambda/D)
100     epsilon_WH = p_WH(1) / 4 * 100; % % (slope = 4*epsilon)
101 else
102     D_WH = NaN;
103     epsilon_WH = NaN;
104     warning('Less than 3 valid peaks for Williamson-Hall.');
```

```

105 end
106
107 % ===== SAVE RESULTS TABLE =====
108 XRD_Table = table(peak_2theta, d_vals, FWHM_rad, D_vals, ...
109     'VariableNames', {'Angle_2theta', 'd_spacing_A', ...
110         'FWHM_rad', 'Crystallite_nm'});
111 writetable(XRD_Table, 'XRD_results.xlsx');
112 writetable(XRD_Table, 'XRD_results.txt', 'Delimiter', '\t');
```

```

113
114 % Save Williamson-Hall results to text file
115 fid = fopen('Williamson_Hall.txt', 'w');
116 fprintf(fid, 'D_WH (nm): %.2f\n', D_WH);
117 fprintf(fid, 'Microstrain (%): %.4f\n', epsilon_WH);
118 fclose(fid);
119
120 % ===== PLOT 1: DIFFRACTOGRAM WITH PEAKS =====
121 fig1 = figure('Position', [100 100 800 500]);
122 plot(two_theta, intensity, 'b-', 'LineWidth', 1.5);
123 hold on;
124 plot(peak_2theta, peak_int, 'ro', 'MarkerSize', 8, 'LineWidth', 2);
125 xlabel('2\theta (degrees)');
126 ylabel('Intensity (a.u.)');
127 title('Diffractiongram with identified peaks');
128 legend('Data', 'Peaks', 'Location', 'northeast');
129 grid on;
130 hold off;
131 exportgraphics(fig1, 'Diffractiongram_Peaks.png', 'Resolution', 300);
132 exportgraphics(fig1, 'Diffractiongram_Peaks.pdf');
```

```

133
134 % ===== PLOT 2: WILLIAMSON-HALL =====
135 if exist('x_WH', 'var') && length(x_WH) >= 3
136     fig2 = figure('Position', [100 100 800 500]);
137     plot(x_WH, y_WH, 'bo', 'MarkerSize', 8, 'MarkerFaceColor', 'b');
138     hold on;
139     x_fit = linspace(min(x_WH), max(x_WH), 100);
140     y_fit = polyval(p_WH, x_fit);
141     plot(x_fit, y_fit, 'r-', 'LineWidth', 2);
142     xlabel('4 sin ');
143     ylabel('cos (rad)');
144     title(sprintf('Williamson-Hall: D = %.2f nm, = %.2f %%', D_WH,
145         epsilon_WH));
145     grid on;
146     hold off;
147     exportgraphics(fig2, 'Williamson_Hall.png', 'Resolution', 300);

```

```

148     exportgraphics(fig2, 'Williamson_Hall.pdf');
149 end
150
151 % ===== PLOT 3: RIETVELD (OBSERVED, CALCULATED, RESIDUALS) =====
152 if exist(rietveld_file, 'file') && exist(bragg_file, 'file')
153     % Load refinement output data
154     riet_data = load(rietveld_file);
155     t2 = riet_data(:, 1);
156     y_obs = riet_data(:, 2);
157     y_calc = riet_data(:, 3);
158     bragg = load(bragg_file);
159
160     % ===== CALCULATE Rwp AND GOF =====
161     % Weight: w_i = 1/Y_obs (Poisson approximation)
162     w = 1 ./ (y_obs + 1e-10); % Avoid division by zero
163     Rwp = sqrt( sum(w .* (y_obs - y_calc).^2) / sum(w .* y_obs.^2) ) *
        100;
164     N = length(y_obs);
165     P = 25; % Number of refined parameters (adjust according to your
        .out file)
166     GOF = sqrt( sum(w .* (y_obs - y_calc).^2) / (N - P) );
167
168     % Create figure with two subplots: data + markers and residuals
169     fig3 = figure('Position', [100 100 900 600]);
170
171     % Upper subplot: Observed, Calculated, and Bragg markers
172     subplot(2,1,1);
173     hold on;
174     plot(t2, y_obs, 'ro', 'MarkerSize', 3, 'MarkerFaceColor', 'r');
175     plot(t2, y_calc, 'b-', 'LineWidth', 1.5);
176     % Add Bragg markers (vertical lines)
177     for i = 1:length(bragg)
178         line([bragg(i) bragg(i)], [0 max(y_obs)*0.1], ...
179             'Color', 'k', 'LineStyle', ':');
180     end
181     xlabel('2\theta (degrees)');
182     ylabel('Intensity');
183     title(sprintf('Rietveld Refinement: R_{wp} = %.2f %%', GOF = %.3f',
        Rwp, GOF));
184     legend('Observed', 'Calculated', 'Bragg', 'Location', 'best');
185     grid on;
186     hold off;
187
188     % Lower subplot: Residuals (Yobs - Ycalc)
189     subplot(2,1,2);
190     plot(t2, y_obs - y_calc, 'g-', 'LineWidth', 1);
191     hold on;
192     line([t2(1) t2(end)], [0 0], 'Color', 'k', 'LineStyle', '--');
193     xlabel('2\theta (degrees)');
194     ylabel('Residuals');
195     grid on;
196     hold off;
197
198     exportgraphics(fig3, 'Rietveld_Plot.png', 'Resolution', 300);
199     exportgraphics(fig3, 'Rietveld_Plot.pdf');
200 end
201
202 disp('XRD analysis completed. Files saved successfully.');
```

Listing 1: MATLAB Code for XRD Complete Analysis

2.4.2 Detailed Code Explanation

Lines 1-4: Clear the environment to avoid conflicts with previous variables.

Lines 7-14: User-configurable settings. `xrd_file` is the data file name. `lambda` depends on the X-ray tube (1.5406 Å for Cu K). `U`, `V`, `W` are Caglioti parameters obtained from a standard measurement (e.g., silicon). `K` is the Scherrer shape constant. `rietveld_file` and `bragg_file` are Rietveld refinement output files.

Lines 17-20: Check if the data file exists. If not, display an error and stop execution.

Lines 23-24: Load data using `load`, which expects a text file with two columns.

Lines 27-31: Detect peaks using `findpeaks`. `MinPeakHeight` filters peaks with intensity less than 20% of the maximum peak. `MinPeakDistance` ensures a minimum separation of 2 degrees between peaks.

Lines 34-36: Preallocate vectors to store results.

Lines 39-67: Loop over each detected peak to fit a Gaussian. This is the heart of the analysis:

- **Window:** Select points within ±1.5 degrees around the peak.
- **Model:** A Gaussian with linear background: $y = a + bx + c \exp(-(x - \mu)^2 / (2\sigma^2))$.
- **Initial guesses:** Based on observed data.
- **Fitting:** Uses `lsqcurvefit` (from Optimization Toolbox) to minimize squared error.
- **FWHM extraction:** From the fitted sigma, calculate FWHM in radians.
- **Instrumental correction:** Use the Caglioti equation to calculate instrumental broadening at each angle and subtract quadratically (for Gaussian profiles).
- ***d*-spacing calculation:** By Bragg's Law.
- ***D* calculation:** By Scherrer equation, using the corrected FWHM.

Lines 70-82: Williamson-Hall:

- Filter peaks with valid FWHM (>0).
- If at least 3 peaks exist, fit a line between $4 \sin \theta$ (x) and $\beta \cos \theta$ (y).
- The intercept gives $K\lambda/D$, and the slope gives 4ϵ .

Lines 85-94: Save results:

- Table with angle, *d*, FWHM (rad), and crystallite size (nm) in Excel and TXT.
- Text file with Williamson-Hall results.

Lines 97-109: Diffractogram plot with marked peaks. Save as PNG (high resolution) and PDF (vector).

Lines 112-121: Williamson-Hall plot with the fitted line.

Lines 124-161: Rietveld plot:

- Read refinement output files.
- Calculate R_{wp} and GOF from the data.
- Plot Observed (red points), Calculated (blue line), Bragg markers (dotted lines), and Residuals.

2.5 Example Input and Output

2.5.1 Input (ZIF-8_400mL.xy)

```
5.00    150
5.05    180
5.10    210
...
70.00   280
```

2.5.2 Output Table XRD_results.xlsx

Table 1: XRD results for ZIF-8

Angle 2	d (Å)	FWHM (rad)	Crystallite (nm)
7.32	12.0645	0.0085	16.42
12.56	7.0452	0.0092	15.10
17.88	4.9563	0.0110	12.68
22.15	4.0101	0.0105	13.45

2.5.3 Output Williamson-Hall

```
D_WH (nm): 14.78
Microstrain (%): 0.0234
```

2.6 Graph Customization for Publication

To obtain publication-quality graphs, adjust the following properties:

```
1 % Set default properties for all figures
2 set(0, 'DefaultAxesFontName', 'Arial');
3 set(0, 'DefaultAxesFontSize', 12);
4 set(0, 'DefaultLineLineWidth', 1.5);
5 set(0, 'DefaultAxesBox', 'on');
6 set(0, 'DefaultAxesGridLineStyle', ':');
7
8 % Example figure with detailed customization
9 fig = figure('Position', [100 100 900 600], 'Color', 'white');
10
11 % Main axes
12 ax1 = axes('Parent', fig, 'Units', 'normalized', 'Position', [0.12 0.12
13     0.75 0.80]);
14 hold(ax1, 'on');
15 % Plotting with colors and styles
```

```

16 h1 = plot(x, y1, 'b-', 'LineWidth', 2, 'DisplayName', 'Data');
17 h2 = plot(x, y2, 'r--', 'LineWidth', 2, 'DisplayName', 'Fit');
18
19 % Axis adjustments
20 xlabel('2\theta (degrees)', 'FontSize', 14, 'FontWeight', 'bold');
21 ylabel('Intensity (a.u.)', 'FontSize', 14, 'FontWeight', 'bold');
22 title('Graph Title', 'FontSize', 16, 'FontWeight', 'bold');
23
24 % Grid and box
25 grid(ax1, 'on');
26 set(ax1, 'GridLineStyle', ':', 'GridColor', [0.5 0.5 0.5]);
27
28 % Custom legend
29 leg = legend([h1, h2], 'Location', 'northeast', 'FontSize', 12);
30 set(leg, 'Box', 'off');
31
32 % Axis limits
33 xlim([min(x) max(x)]);
34 ylim([0 max(y1)*1.1]);
35
36 % Save in high resolution
37 exportgraphics(fig, 'custom_figure.png', 'Resolution', 600);
38 exportgraphics(fig, 'custom_figure.pdf');

```

Listing 2: Example of advanced graph customization

Customization Options:

Table 2: Main graph properties in MATLAB

Property	Parameter	Example
Line color	Color	'r', [0.5 0.2 0.8]
Line style	LineStyle	'-', '--', ':'
Line width	LineWidth	1.5, 2, 3
Marker	Marker	'o', 's', 'd'
Marker size	MarkerSize	6, 8, 10
Marker color	MarkerFaceColor	'b', 'r'
Font size	FontSize	12, 14, 16
Font name	FontName	'Arial', 'Times'
Legend location	Location	'northeast', 'southwest'
Grid	grid	on, off

References - XRD and Rietveld

1. Bragg, W. H.; Bragg, W. L. The Reflection of X-Rays by Crystals. *Proc. R. Soc. Lond. A* **1913**, 88, 428-438.
2. Scherrer, P. Bestimmung der Grösse und der inneren Struktur von Kolloidteilchen mittels Röntgenstrahlen. *Nachr. Ges. Wiss. Göttingen* **1918**, 2, 98-100.

3. Caglioti, G.; Paoletti, A.; Ricci, F. P. Choice of Collimators for a Crystal Spectrometer for Neutron Diffraction. *Nucl. Instrum.* **1958**, 3, 223-228.
4. Rietveld, H. M. A Profile Refinement Method for Nuclear and Magnetic Structures. *J. Appl. Crystallogr.* **1969**, 2, 65-71.
5. Young, R. A. *The Rietveld Method*; Oxford University Press: Oxford, 1993.
6. Toby, B. H. EXPGUI, a graphical user interface for GSAS. *J. Appl. Crystallogr.* **2001**, 34, 210-213.

3 BET Analysis (Specific Surface Area)

3.1 Theoretical Foundations

The determination of specific surface area by gas adsorption is one of the most important techniques for characterizing porous materials. The BET (Brunauer-Emmett-Teller) method, proposed in 1938, extends the Langmuir model to multilayer adsorption and is the standard for surface area determination.

3.1.1 BET Equation

The linearized BET equation is:

$$\frac{P/P_0}{V(1 - P/P_0)} = \frac{1}{V_m C} + \frac{C - 1}{V_m C} \cdot (P/P_0) \quad (10)$$

where:

- P/P_0 is the relative pressure
- V is the volume of gas adsorbed (cm³/g, at STP)
- V_m is the volume of gas required to form a monolayer
- C is the BET constant (related to adsorption energy)

Defining $x = P/P_0$ and $y = x/[V(1 - x)]$, the equation becomes a line:

$$y = a \cdot x + b \quad (11)$$

where $a = (C - 1)/(V_m C)$ and $b = 1/(V_m C)$.

3.1.2 BET Parameters

From the slope a and intercept b :

$$V_m = \frac{1}{a + b}, \quad C = \frac{a}{b} + 1 \quad (12)$$

3.1.3 Specific Surface Area

The surface area is calculated from the monolayer volume:

$$S_{BET} = \frac{V_m \cdot N_A \cdot A_m}{V_{mol}} \quad (13)$$

where:

- $N_A = 6.022 \times 10^{23}$ mol⁻¹ (Avogadro's number)
- $A_m = 16.2 \times 10^{-20}$ m² (molecular area of N at 77 K)
- $V_{mol} = 22414$ cm³/mol (molar volume at STP)

3.1.4 Validity Range and Rouquerol Criteria

The BET theory is valid only at low relative pressures. The usual range is:

- Mesoporous materials: $0.05 < P/P_0 < 0.35$
- Microporous materials: $0.01 < P/P_0 < 0.15$

Two criteria (Rouquerol) must be met for the fit to be considered valid:

1. $C > 0$ (positive intercept)
2. $V(0.05) < V_m$ (volume adsorbed at $P/P_0 = 0.05$ is less than the monolayer volume)

3.2 MATLAB Code for BET With Detailed Explanations

```

1  %% =====
2  % BET ANALYSIS - SURFACE AREA, C AND VALIDATION
3  % =====
4
5  clear; clc; close all;
6
7  % ===== CONFIGURATIONS =====
8  bet_file = 'ZIF-8_400mL_isotherm.txt';
9  x_min = 0.05; % Lower limit of fitting interval
10 x_max = 0.35; % Upper limit of fitting interval
11 % =====
12
13 % Check if file exists
14 if ~exist(bet_file, 'file')
15     error('File %s not found.', bet_file);
16 end
17
18 % Load data: two columns (P/P0, Adsorbed volume)
19 data = load(bet_file);
20 P_P0 = data(:, 1);
21 V = data(:, 2);
22
23 % Filter valid data (P/P0 > 0 and V > 0)
24 valid = (P_P0 > 0) & (V > 0);
25 x = P_P0(valid);
26 V_ads = V(valid);
27
28 % ===== LINEAR TRANSFORMATION =====
29 % y = x / (V * (1 - x))
30 y = x ./ (V_ads .* (1 - x));
31
32 % Select fitting interval
33 idx = (x >= x_min) & (x <= x_max);
34 x_fit = x(idx);
35 y_fit = y(idx);
36
37 % Check if there are enough points
38 if length(x_fit) < 3
39     error('Too few points in the selected interval. Adjust x_min and
40         x_max.');
```

```

43 % Weights = 1/x to correct heteroscedasticity (non-constant variance)
44 weights = 1 ./ (x_fit + 1e-10);
45 W = diag(weights);
46 X = [x_fit, ones(size(x_fit))];
47 Y = y_fit;
48 coeffs = (X' * W * X) \ (X' * W * Y); % WLS solution
49 a = coeffs(1);
50 b = coeffs(2);
51
52 % ===== BET PARAMETERS CALCULATION =====
53 Vm = 1 / (a + b);
54 C = (a / b) + 1;
55
56 % Physical constants
57 NA = 6.022e23; % mol^-1
58 Am = 16.2e-20; % m^2 (N2 molecular area)
59 Vmol = 22414; % cm^3/mol
60 S_BET = (Vm * NA * Am) / Vmol; % m^2/g
61
62 % ===== WEIGHTED R^2 =====
63 y_pred = a*x_fit + b;
64 R2 = 1 - sum(weights .* (y_fit - y_pred).^2) / sum(weights .* (y_fit -
    mean(y_fit)).^2);
65
66 % ===== ROUQUEROL CRITERIA =====
67 C_valid = (C > 0);
68 % Find V(0.05)
69 idx_005 = find(x >= 0.05, 1, 'first');
70 if ~isempty(idx_005)
71     V_005 = V_ads(idx_005);
72     V_less_than_Vm = (V_005 < Vm);
73 else
74     V_less_than_Vm = false;
75 end
76
77 if ~(C_valid && V_less_than_Vm)
78     warning('BET fit invalid by Rouquerol criteria.');
```

fprintf('C = %.2f, V(0.05) = %.4f, Vm = %.4f\n', C, V_005, Vm);

```

80 end
81
82 % ===== SAVE RESULTS =====
83 BET_Table = table(x_min, x_max, Vm, C, S_BET, R2, ...
84     'VariableNames', {'x_min', 'x_max', 'Vm_cm3_g', 'C', ...
85         'S_BET_m2_g', 'R2'});
86 writetable(BET_Table, 'BET_results.xlsx');
87 writetable(BET_Table, 'BET_results.txt', 'Delimiter', '\t');
88
89 % ===== PLOTS =====
90 fig = figure('Position', [100 100 900 500]);
91
92 % Subplot 1: Adsorption isotherm
93 subplot(1,2,1);
94 plot(P_P0, V, 'bo-', 'LineWidth', 1.5, 'MarkerFaceColor', 'b',
    'MarkerSize', 6);
95 hold on;
96 % Highlight the fitting interval
97 x_fill = [x_min, x_max, x_max, x_min];
98 y_fill = [0, 0, max(V)*1.1, max(V)*1.1];

```

```

99 fill(x_fill, y_fill, 'r', 'FaceAlpha', 0.1, 'EdgeColor', 'none');
100 xlabel('P/P_0', 'FontSize', 12);
101 ylabel('Volume (cm³/g)', 'FontSize', 12);
102 title('Adsorption Isotherm', 'FontSize', 14);
103 grid on;
104 hold off;
105
106 % Subplot 2: Linearized BET plot
107 subplot(1,2,2);
108 plot(x, y, 'bo', 'MarkerSize', 6, 'MarkerFaceColor', 'b');
109 hold on;
110 plot(x_fit, y_pred, 'r-', 'LineWidth', 2);
111 xlabel('P/P_0', 'FontSize', 12);
112 ylabel('(P/P_0)/[V(1-P/P_0)]', 'FontSize', 12);
113 title(sprintf('BET: S = %.2f m²/g, R² = %.4f', S_BET, R2), 'FontSize',
114         14);
115 legend('Data', 'WLS Fit', 'Location', 'northwest', 'FontSize', 10);
116 grid on;
117 hold off;
118
119 % Export figure
120 exportgraphics(fig, 'BET_Plot.png', 'Resolution', 300);
121 exportgraphics(fig, 'BET_Plot.pdf');
122
123 disp('BET analysis completed. Files saved successfully.');
```

Listing 3: MATLAB Code for BET with statistical validation

3.3 Detailed Code Explanation for BET

Lines 1-4: Clear the environment.

Lines 7-10: Configurations: data file and fitting interval.

Lines 13-16: Check if the file exists.

Lines 19-23: Load data and filter invalid values (zero pressure or negative volume).

Lines 26-27: Apply BET linear transformation: $y = x/[V(1 - x)]$.

Lines 30-33: Select only points within the fitting interval.

Lines 36-42: Weighted linear regression:

- Weights $1/x$ correct heteroscedasticity (residual variance increases with x).
- The solution is obtained by weighted least squares (WLS) using the normal equation $(X^T W X)^{-1} X^T W Y$.

Lines 45-47: Calculate V_m and C from a and b .

Lines 50-53: Calculate surface area S_{BET} using physical constants.

Lines 56-57: Calculate weighted R^2 , which is more appropriate for data with non-constant variance.

Lines 60-72: Apply Rouquerol criteria to validate the fit. If $C < 0$ or $V(0.05) > V_m$, issue a warning.

Lines 75-78: Save the results table in Excel and TXT.

Lines 81-101: Generate two plots:

- Adsorption isotherm with the fitting interval highlighted.
- Linearized BET plot with the fitted line.

3.4 Example Input and Output

3.4.1 Input (ZIF-8_400mL_isotherm.txt)

```
0.010    5.20
0.050    25.10
0.100    48.30
0.150    68.50
0.200    85.20
0.250    99.80
0.300   112.50
0.350   123.70
0.400   133.20
0.500   148.50
```

3.4.2 Output Table BET_results.xlsx

Table 3: BET results for ZIF-8

x_min	x_max	Vm (cm ³ /g)	C	S_BET (m ² /g)	R ²
0.05	0.35	12.3456	85.72	48.27	0.9998

References - BET

1. Brunauer, S.; Emmett, P. H.; Teller, E. Adsorption of Gases in Multimolecular Layers. *J. Am. Chem. Soc.* **1938**, 60, 309-319.
2. Rouquerol, J.; Llewellyn, P.; Rouquerol, F. Is the BET Equation Applicable to Microporous Adsorbents? *Stud. Surf. Sci. Catal.* **2007**, 160, 49-56.
3. Thommes, M.; Kaneko, K.; Neimark, A. V.; Olivier, J. P.; Rodriguez-Reinoso, F.; Rouquerol, J.; Sing, K. S. W. Physisorption of Gases, with Special Reference to the Evaluation of Surface Area and Pore Size Distribution. *Pure Appl. Chem.* **2015**, 87, 1051-1069.

4 TGA-DTA/DTG Analysis (Thermogravimetry)

4.1 Theoretical Foundations

Thermogravimetric analysis (TGA) measures the mass change of a sample as a function of temperature or time, under a controlled atmosphere. It is widely used to study thermal decomposition, stability, composition, and reaction kinetics.

4.1.1 Mass Loss

The mass loss in each decomposition step is calculated as:

$$\Delta m(\%) = \frac{m_{\text{initial}} - m_{\text{final}}}{m_{\text{initial}}} \times 100 \quad (14)$$

Important: The loss must be calculated from the **plateaus** of the TGA curve, never from the DTG peak value. The DTG peak corresponds to the inflection point, where the loss rate is maximum, but the mass is still decreasing.

4.1.2 DTG (Derivative of TGA)

The DTG is the first derivative of the TGA curve:

$$DTG = \frac{dm}{dT} \quad \text{or} \quad \frac{dm}{dt} \quad (15)$$

Negative peaks in the DTG indicate the temperature of maximum decomposition rate (T_{peak}), also called the peak temperature.

4.1.3 Onset Temperature

The onset temperature is where decomposition begins. It is defined as the intersection of the baseline with the tangent to the TGA curve at the inflection point (where DTG is maximum).

4.1.4 Decomposition Kinetics Kissinger Method

The Kissinger method is a model-free approach to calculate the apparent activation energy (E_a) from experiments conducted at different heating rates (β):

$$\ln \left(\frac{\beta}{T_p^2} \right) = -\frac{E_a}{R} \cdot \frac{1}{T_p} + \text{constant} \quad (16)$$

Fitting a line between $\ln(\beta/T_p^2)$ (y-axis) and $1/T_p$ (x-axis), the slope gives $-E_a/R$.

4.2 MATLAB Code for TGA With Detailed Explanations

```

1 %% =====
2 %   TGA-DTA/DTG ANALYSIS - PLATEAU LOSSES, ONSET AND KISSINGER
3 %   =====
4
5 clear; clc; close all;
```

```

6
7 % ===== CONFIGURATIONS =====
8 tga_file = 'ZIF-8_400mL_TGA.txt';
9 min_height = 0.5; % Minimum height for DTG peak detection
10 min_distance = 20; % Minimum distance between peaks (in degrees
    Celsius)
11 % Kissinger data (provided manually by user)
12 % beta_list = [5, 10, 15, 20]; % Heating rates (°C/min)
13 % Tp_list = [350, 365, 375, 382]; % Peak temperatures (°C)
14 % =====
15
16 % Check if file exists
17 if ~exist(tga_file, 'file')
18     error('File %s not found.', tga_file);
19 end
20
21 % Load data: three columns (Temperature, Mass, DTA)
22 data = load(tga_file);
23 Temp = data(:, 1);
24 Mass = data(:, 2);
25 DTA = data(:, 3);
26
27 % ===== SMOOTHING AND DERIVATIVE =====
28 % Savitzky-Golay filter (order 3, window 11 points)
29 Mass_smooth = sgolayfilt(Mass, 3, 11);
30 % Calculate DTG (derivative of mass with respect to temperature)
31 DTG = gradient(Mass_smooth, Temp);
32
33 % ===== PEAK DETECTION IN DTG =====
34 % findpeaks detects minima (using -DTG) with minimum height and distance
    in degrees
35 [peak_value, peak_idx] = findpeaks(-DTG, Temp, ...
36     'MinPeakHeight', min_height, ...
37     'MinPeakDistance', min_distance);
38 DTG_peak = -peak_value;
39 T_peak = Temp(peak_idx);
40 Mass_peak = Mass_smooth(peak_idx);
41
42 % ===== CORRECT PLATEAU LOSS CALCULATION =====
43 % Define a threshold to consider DTG ~ 0 (plateau)
44 dtg_threshold = 0.01 * max(abs(DTG));
45
46 if isempty(peak_idx)
47     warning('No DTG peaks detected.');
```

losses = [];

```

48 else
49     plateau_masses = [];
50
51     % Initial plateau (before first peak)
52     % Find the last point before the peak where DTG > threshold
53     idx_before = find(abs(DTG(1:peak_idx(1))) > dtg_threshold, 1,
54         'last');
```

if isempty(idx_before)

```

55     idx_before = 1;
56 end
57 plateau_masses = [plateau_masses;
58     mean(Mass_smooth(max(1, idx_before-2):min(end, idx_before+2)))];
59
```

```

60 % Plateaus between consecutive peaks
61 for i = 1:length(peak_idx)-1
62     % Find point where DTG crosses the threshold between peak i and
        i+1
63     interval = peak_idx(i):peak_idx(i+1);
64     idx_cross = find(abs(DTG(interval)) < dtg_threshold, 1, 'first');
65     if isempty(idx_cross)
66         % If not found, use midpoint
67         idx_cross = round(mean([peak_idx(i), peak_idx(i+1)]));
68     else
69         idx_cross = interval(idx_cross);
70     end
71     plateau_masses = [plateau_masses;
        mean(Mass_smooth(max(1,idx_cross-2):min(end,idx_cross+2)))];
72 end
73
74 % Final plateau (after last peak)
75 idx_after = find(abs(DTG(peak_idx(end):end)) > dtg_threshold, 1,
    'first');
76 if isempty(idx_after)
77     idx_after = length(Temp);
78 else
79     idx_after = peak_idx(end) + idx_after - 1;
80 end
81 plateau_masses = [plateau_masses;
    mean(Mass_smooth(max(1,idx_after-2):min(end,idx_after+2)))];
82
83 % Calculate losses (%) as difference between consecutive plateaus
84 losses = -diff(plateau_masses);
85 end
86
87 % ===== SAVE LOSSES TABLE =====
88 if ~isempty(losses)
89     TGA_Table = table(T_peak, losses, ...
90         'VariableNames', {'T_peak_C', 'Mass_loss_percentage'});
91     writetable(TGA_Table, 'TGA_loss_results.xlsx');
92 else
93     TGA_Table = table();
94 end
95
96 % ===== ONSET CALCULATION (first peak) =====
97 if ~isempty(T_peak)
98     idx_first = peak_idx(1);
99     % Window of 45 degrees around the inflection point
100     margin = 5; % degrees
101     idx_start = find(Temp >= Temp(idx_first)-margin, 1, 'first');
102     idx_end = find(Temp <= Temp(idx_first)+margin, 1, 'last');
103     if isempty(idx_start), idx_start = 1; end
104     if isempty(idx_end), idx_end = length(Temp); end
105     range = idx_start:idx_end;
106
107     % Linear fit in the region (tangent)
108     p_tang = polyfit(Temp(range), Mass_smooth(range), 1);
109
110     % Baseline: mean of first 5% of points
111     base_idx = 1:round(0.05*length(Temp));
112     base_mean = mean(Mass_smooth(base_idx));
113

```



```

114     % Onset: intersection of tangent with baseline
115     T_onset = (base_mean - p_tang(2)) / p_tang(1);
116 else
117     T_onset = NaN;
118 end
119
120 % ===== SAVE ONSET AND RESIDUE =====
121 fid = fopen('TGA_onset_results.txt', 'w');
122 fprintf(fid, 'T_onset (C): %.2f\n', T_onset);
123 fprintf(fid, 'Final_residue (%): %.2f\n', Mass_smooth(end));
124 fclose(fid);
125
126 % ===== KISSINGER (if data provided) =====
127 if exist('beta_list', 'var') && exist('Tp_list', 'var') &&
    length(beta_list) >= 3
128     Tp_K = Tp_list + 273.15; % Convert to Kelvin
129     x_K = 1 ./ Tp_K;
130     y_K = log(beta_list ./ Tp_K.^2);
131     p_K = polyfit(x_K, y_K, 1);
132     Ea_kJ = -p_K(1) * 8.314 / 1000; % kJ/mol
133
134     Kissinger_Table = table(beta_list', Tp_list', ...
135         'VariableNames', {'Heating_rate_C_min', 'T_peak_C'});
136     writetable(Kissinger_Table, 'Kissinger_results.xlsx');
137     fid2 = fopen('Kissinger_results.txt', 'w');
138     fprintf(fid2, 'Ea (kJ/mol): %.2f\n', Ea_kJ);
139     fclose(fid2);
140 end
141
142 % ===== PLOTS =====
143 fig = figure('Position', [100 100 900 650]);
144
145 % Subplot 1: TGA + DTG
146 subplot(2,1,1);
147 yyaxis left;
148 plot(Temp, Mass_smooth, 'b-', 'LineWidth', 2);
149 ylabel('Mass (%)', 'FontSize', 12);
150 ylim([0 105]);
151 hold on;
152 if ~isempty(T_peak)
153     plot(T_peak, Mass_peak, 'ro', 'MarkerSize', 8, 'LineWidth', 2);
154 end
155 if ~isnan(T_onset)
156     plot(T_onset, base_mean, 'go', 'MarkerSize', 10, 'LineWidth', 2);
157     % Draw the tangent
158     T_tang = [T_onset-20, T_onset+20];
159     M_tang = p_tang(1)*T_tang + p_tang(2);
160     plot(T_tang, M_tang, 'g--', 'LineWidth', 1.5);
161 end
162 yyaxis right;
163 plot(Temp, DTG, 'k-', 'LineWidth', 1.5);
164 ylabel('DTG (%/°C)', 'FontSize', 12);
165 xlabel('Temperature (°C)', 'FontSize', 12);
166 title('TGA and DTG', 'FontSize', 14);
167 legend('TGA', 'T_{peak}', 'T_{onset}', 'Tangent', 'DTG', ...
168     'Location', 'best', 'FontSize', 10);
169 grid on;
170 hold off;

```

```

171
172 % Subplot 2: DTA
173 subplot(2,1,2);
174 plot(Temp, DTA, 'r-', 'LineWidth', 2);
175 xlabel('Temperature (řC)', 'FontSize', 12);
176 ylabel('DTA (řV)', 'FontSize', 12);
177 title('DTA', 'FontSize', 14);
178 grid on;
179
180 % Export figure
181 exportgraphics(fig, 'TGA_DTA_DTG.png', 'Resolution', 300);
182 exportgraphics(fig, 'TGA_DTA_DTG.pdf');
183
184 % ===== KISSINGER PLOT =====
185 if exist('x_K', 'var')
186     figK = figure('Position', [100 100 800 500]);
187     plot(x_K, y_K, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
188     hold on;
189     x_fitK = linspace(min(x_K), max(x_K), 100);
190     y_fitK = polyval(p_K, x_fitK);
191     plot(x_fitK, y_fitK, 'b-', 'LineWidth', 2);
192     xlabel('1/T_p (1/K)', 'FontSize', 12);
193     ylabel('ln(\beta / T_p^2)', 'FontSize', 12);
194     title(sprintf('Kissinger: E_a = %.2f kJ/mol', Ea_kJ), 'FontSize',
195           14);
196     grid on;
197     hold off;
198     exportgraphics(figK, 'Kissinger_Plot.png', 'Resolution', 300);
199     exportgraphics(figK, 'Kissinger_Plot.pdf');
200 end
201 disp('TGA analysis completed. Files saved successfully.');
```

Listing 4: MATLAB Code for TGA with correct plateau loss calculation

4.3 Detailed Code Explanation for TGA

Lines 1-4: Clear the environment.

Lines 7-12: Configurations: data file, peak detection parameters, and optional Kissinger data.

Lines 15-18: Check if the file exists.

Lines 21-24: Load data: three columns (Temperature, Mass, DTA).

Lines 27-30: Smooth the mass curve using Savitzky-Golay filter (reduces noise without distorting peaks) and calculate DTG using gradient.

Lines 33-37: Detect peaks in DTG using `findpeaks(-DTG)`. The `MinPeakHeight` and `MinPeakDistance` parameters filter spurious peaks.

Lines 40-77: Correct plateau loss calculation:

- Define a threshold for when DTG is approximately zero (plateau).
- For each peak, find the plateau before and after, using DTG intersection with the threshold.

- Calculate the average mass at each plateau.
- Losses are the negative difference between consecutive plateaus.

Lines 80-83: Save the losses table.

Lines 86-101: Calculate onset temperature:

- Select a window of ± 5 degrees around the inflection point (first peak).
- Fit a line (tangent) in this region.
- Calculate the baseline as the mean of the first 5% of data points.
- The onset is the intersection of the tangent with the baseline.

Lines 104-107: Save onset and final residue.

Lines 110-120: Kissinger method: if the user provides heating rates and peak temperatures, calculate the activation energy.

Lines 123-159: Generate plots:

- TGA with DTG (dual axes) showing peaks, onset, and tangent.
- DTA separately.
- Kissinger plot (if applicable).

4.4 Example Input and Output

4.4.1 Input (ZIF-8_400mL_TGA.txt)

```
25.0    100.00    0.05
30.0    99.95    0.06
35.0    99.90    0.05
...
800.0   42.00    0.02
```

4.4.2 Output Table TGA_loss_results.xlsx

Table 4: TGA results for ZIF-8

T _{peak} (°C)	Mass loss (%)
150.2	12.45
380.5	28.73
520.1	15.82

4.4.3 Output TGA_onset_results.txt

```
T_onset (C): 142.30
Final_residue (%): 42.85
```

References - TGA and Kinetics

1. Kissinger, H. E. Reaction Kinetics in Differential Thermal Analysis. *Anal. Chem.* **1957**, 29, 1702-1706.
2. ASTM E1131-20. *Standard Test Method for Compositional Analysis by Thermogravimetry*; ASTM International, 2020.
3. Brown, M. E. *Introduction to Thermal Analysis: Techniques and Applications*; Springer, 2001.
4. Vyazovkin, S.; Burnham, A. K.; Criado, J. M.; Pérez-Maqueda, L. A.; Popescu, C.; Sbirrazzuoli, N. ICTAC Kinetics Committee Recommendations for Performing Kinetic Computations on Thermal Analysis Data. *Thermochim. Acta* **2011**, 520, 1-19.

5 Multiple File Plotting and Customization

5.1 Example Code for Overlay Plotting

```

1  % =====
2  % PLOTTING MULTIPLE DIFFRACTOGRAMS
3  % =====
4
5  % List of files, colors, styles, and legends
6  files = {'ZIF-8_400mL.xy', 'ZIF-67_400mL.xy', 'ZIF-8_600mL.xy'};
7  colors = {'#E63946', '#2A9D8F', '#264653'}; % Hexadecimal colors
8  styles = {'-', '--', '-.'};
9  widths = [2, 1.8, 1.8];
10 legends = {'ZIF-8 400mL', 'ZIF-67 400mL', 'ZIF-8 600mL'};
11
12 % Create figure with appropriate size for publication
13 fig = figure('Position', [100 100 900 600], 'Color', 'white');
14 hold on;
15
16 % Loop over files
17 for i = 1:length(files)
18     if ~exist(files{i}, 'file')
19         warning('File %s not found.', files{i});
20         continue;
21     end
22     data = load(files{i});
23     plot(data(:,1), data(:,2), 'Color', colors{i}, ...
24          'LineStyle', styles{i}, 'LineWidth', widths(i), ...
25          'DisplayName', legends{i});
26 end
27
28 % Axis customization
29 xlabel('2\theta (degrees)', 'FontSize', 14, 'FontWeight', 'bold');
30 ylabel('Intensity (a.u.)', 'FontSize', 14, 'FontWeight', 'bold');
31 title('Diffraction Comparison', 'FontSize', 16, 'FontWeight', 'bold');
32
33 % Legend
34 legend('Location', 'northeast', 'FontSize', 12, 'Box', 'off');
35
36 % Grid and box
37 grid on;
38 set(gca, 'GridLineStyle', ':', 'GridColor', [0.6 0.6 0.6]);
39 box on;
40
41 % Axis limits
42 xlim([min(data(:,1)) max(data(:,1))]);
43 ylim([0 max(ylim)*1.05]);
44
45 % Export in high resolution
46 exportgraphics(gcf, 'XRD_Comparison.png', 'Resolution', 600);
47 exportgraphics(gcf, 'XRD_Comparison.pdf');
48
49 hold off;

```

Listing 5: Plotting multiple diffractograms

5.2 Advanced Customization Table

Table 5: Properties for publication-quality graphs

Property	Parameter/Value	Example
Default font	<code>set(0, 'DefaultAxesFontName', 'Arial')</code>	Arial, Times, Helvetica
Font size	<code>set(0, 'DefaultAxesFontSize', 11)</code>	10, 11, 12
Figure color	<code>set(gcf, 'Color', 'white')</code>	white, [1 1 1]
Figure dimensions	<code>set(gcf, 'Position', [100 100 900 600])</code>	Width x Height (px)
Line color (RGB)	<code>'Color', [0.2 0.4 0.8]</code>	1x3 vector between 0 and 1
Line color (hex)	<code>'Color', '#2A9D8F'</code>	Hexadecimal code
Line width	<code>'LineWidth', 1.5</code>	1.0, 1.5, 2.0
Grid	<code>grid on;</code> <code>set(gca, 'GridLineStyle', ':')</code>	:-, -, -
Box	<code>box on</code>	on, off
Borderless legend	<code>legend('Location', 'best', 'Box', 'off')</code>	off, on
PNG resolution	<code>'Resolution', 600</code>	300, 600, 1200
Vector format	<code>exportgraphics(gcf, 'figure.pdf')</code>	PDF, EPS

5.3 Tips for Publication-Quality Graphs

1. **Use vector fonts:** Arial or Helvetica are preferred for readability at different sizes.
2. **Colors:** Use contrasting and accessible color palettes (e.g., ColorBrewer).
3. **Line thickness:** 1.52 pt lines are ideal for single-column figures.
4. **Resolution:** For PNG, 600 dpi is the minimum for printing. Prefer PDF (vector) whenever possible.
5. **Legend size:** Use 80-90% of the axis font size.
6. **Zoomed insets:** Use `axes('Position', [x, y, w, h])` to create sub-graphs that magnify specific regions while maintaining proportion and clarity.

A Summary of Generated Files

Table 6: Files generated by each analysis

Technique	Tables	Figures
XRD	XRD_results.xlsx / .txt	Diffractogram_Peaks.png / .pdf
	Williamson_Hall.txt	Williamson_Hall.png / .pdf Rietveld_Plot.png / .pdf
BET	BET_results.xlsx / .txt	BET_Plot.png / .pdf
TGA	TGA_loss_results.xlsx	TGA_DTA_DTG.png / .pdf
	TGA_onset_results.txt	Kissinger_Plot.png / .pdf
	Kissinger_results.xlsx / .txt	